

Lecture 3.1 — Scan Conversion II: Circles, Ellipses, Rectangle Filling, Region Filling, and Character Rasterization

Learning outcomes

1. Explain why circle scan conversion needs a special algorithm.
 2. Use the symmetry of a circle to reduce computation.
 3. Explain the midpoint circle algorithm in an intuitive way.
 4. Relate Bresenham's circle algorithm to midpoint decision making.
 5. Explain, at least conceptually, how ellipse scan conversion works.
 6. Explain how rectangles are rasterized efficiently.
 7. Distinguish **boundary fill**, **flood fill**, and **scan-line fill**.
 8. Explain how a character or glyph is scan converted.
 9. Connect these topics to fonts, interfaces, painting tools, and CAD systems.
-

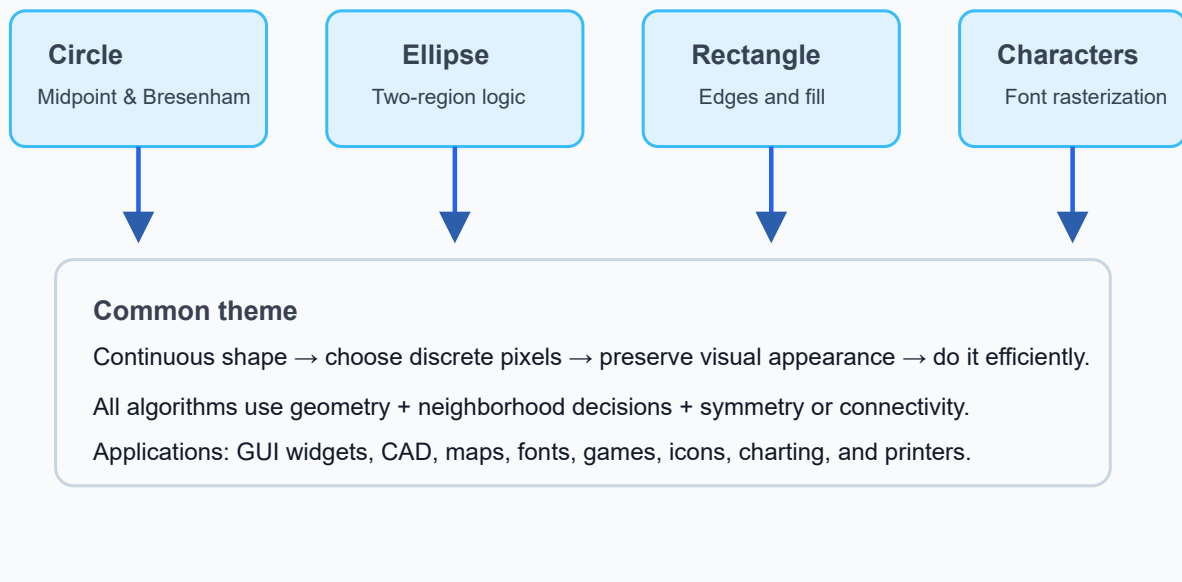
1. Why this lecture matters

In the previous scan conversion lecture, we focused mainly on **points and lines**. But real graphics systems also need to draw:

- circles for knobs, clocks, gauges, icons, and game objects,
- ellipses for orbits, buttons, logos, and perspective effects,
- rectangles for windows, buttons, panels, and UI layout,
- filled regions for paint programs and polygon interiors,
- characters for all text shown on screen.

So this lecture is a natural extension of Lecture 3.

Scan Conversion II at a Glance



A useful high-level idea is this:

Every scan conversion algorithm takes a **continuous geometric description** and turns it into a **finite set of pixels** that looks visually correct.

The only difference is the geometry of the object and the strategy used to pick pixels.

2. Circle scan conversion: the basic challenge

A mathematical circle centered at the origin with radius r is described by:

$$x^2 + y^2 = r^2$$

This equation describes infinitely many points. A raster display, however, needs a finite set of pixels.

The challenge is:

Which pixels should be selected so that they look like a circle?

A naive approach would be to solve for y as:

$$y = \pm \sqrt{r^2 - x^2}$$

and then round the value. This works in principle, but it is not the best approach because:

- square root is computationally expensive,
- floating-point arithmetic is slower than simple integer updates,
- we want a method that picks the next pixel using only small local decisions.

This is why the **midpoint circle algorithm** and **Bresenham's circle algorithm** are so important.

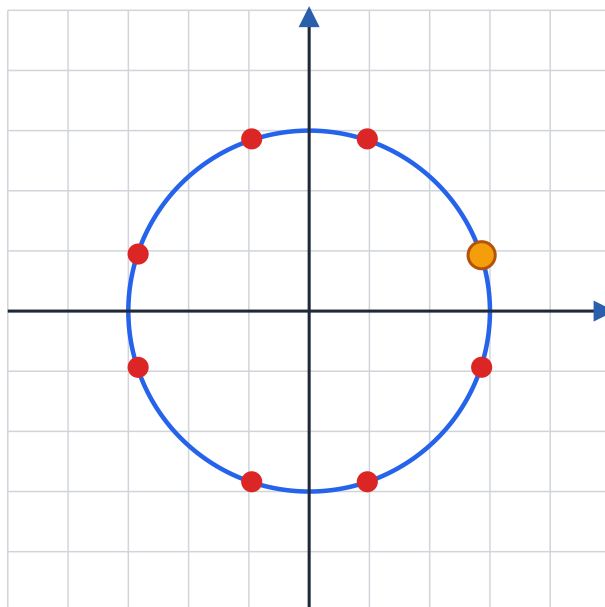
Trivia

Early graphics systems and plotters had very limited hardware. Efficient algorithms mattered a lot because every arithmetic operation was precious. This is one reason why classic raster algorithms are still taught: they show how smart mathematics can save computation.

3. The key trick: symmetry of a circle

A circle is highly symmetric. If we know one point (x, y) on the circle, then we automatically know seven more symmetric points.

Eight-way symmetry of a circle



Compute one octant only
Then reflect the point into
7 other symmetric positions.

Example:

If one point is (x, y) , then
the others are $(\pm x, \pm y)$ and $(\pm y, \pm x)$.

This symmetry is what makes circle
algorithms so efficient.

From one point (x, y) in the first octant, the full set of symmetric points is:

```
( x,  y)
( y,  x)
(-x,  y)
(-y,  x)
```

```
(-x, -y)
(-y, -x)
( x, -y)
( y, -x)
```

So instead of drawing the entire circle, we can:

1. compute points only in **one octant**, and
2. reflect those points into the other 7 octants.

This reduces the work dramatically.

Why octants?

A full circle has 360° . Because of symmetry, it is enough to compute only 45° worth of points, then mirror them.

That means we often stop when:

$$x \geq y$$

At that point we have crossed the 45° line.

4. Midpoint circle algorithm — intuition first

The midpoint circle algorithm is one of the nicest examples in computer graphics because the idea is simple and geometric.

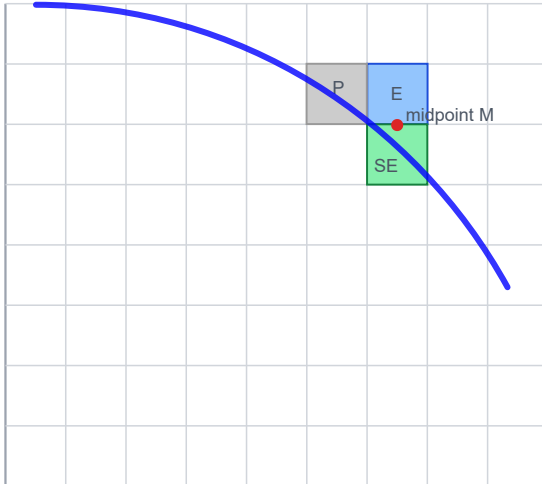
Suppose we are in the first octant and currently at pixel (x, y) .

When we move to the next x-position, the next circle pixel must be one of two nearby candidates:

- E = $(x + 1, y)$
- SE = $(x + 1, y - 1)$

The question is: which one is closer to the true circle?

Midpoint circle decision idea



At each step, x increases by 1 pixel.

We must choose between two candidates:

- $E \rightarrow (x+1, y)$
- $SE \rightarrow (x+1, y-1)$

Check the midpoint M between them.

If M is inside the circle, choose E .

If M is outside the circle, choose SE .

Decision function:

$$f(x, y) = x^2 + y^2 - r^2$$

Instead of comparing both candidate distances directly, we look at the **midpoint** between them:

$$M = (x + 1, y - 1/2)$$

Now define the circle function:

$$f(x, y) = x^2 + y^2 - r^2$$

This function tells us whether a point is:

- **inside** the circle $\rightarrow f(x, y) < 0$
- **on** the circle $\rightarrow f(x, y) = 0$
- **outside** the circle $\rightarrow f(x, y) > 0$

So we test the midpoint M .

- If $f(M) < 0$, the midpoint is inside the circle, so the true circle is closer to **E**.
- If $f(M) > 0$, the midpoint is outside the circle, so the true circle is closer to **SE**.

That is the core idea.

The initial decision parameter

At the start, we take the point:

$$(x_0, y_0) = (0, r)$$

The first midpoint is:

$$(1, r - 1/2)$$

The standard initial decision parameter becomes:

$$p_0 = 1 - r$$

This compact formula is one reason the algorithm is so elegant.

5. Midpoint circle algorithm — easy update logic

We start from:

$$\begin{aligned}x &= 0 \\y &= r \\p &= 1 - r\end{aligned}$$

Then repeat while $x < y$:

- increment x
- if $p < 0$, choose **E**
- otherwise choose **SE** and decrement y
- update p

Update rules

If we choose **E**:

$$\begin{aligned}x &= x + 1 \\p &= p + 2x + 1\end{aligned}$$

If we choose **SE**:

$$\begin{aligned}x &= x + 1 \\y &= y - 1 \\p &= p + 2x + 1 - 2y\end{aligned}$$

Different books may write the update using slightly different indexing conventions, but the logic is the same.

Pseudocode

```
MidpointCircle(r):  
    x = 0  
    y = r  
    p = 1 - r  
  
    while x <= y:  
        plot8(x, y)  
        x = x + 1  
  
        if p < 0:  
            p = p + 2*x + 1  
        else:  
            y = y - 1  
            p = p + 2*x + 1 - 2*y
```

Here `plot8(x, y)` means: plot all 8 symmetric points.

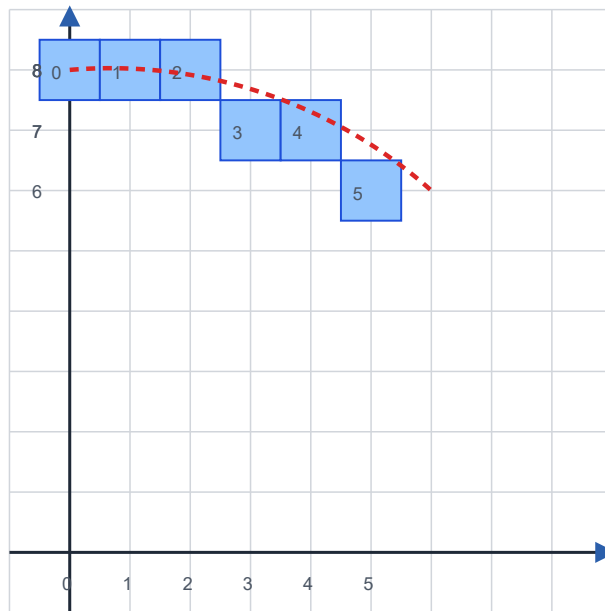
6. Worked example: midpoint circle for `r = 8`

Let us compute the first-octant points for radius `r = 8`.

Start with:

```
x = 0  
y = 8  
p = 1 - 8 = -7
```

Worked example: first-octant points for $r = 8$



k	(x,y)	p_k	Choice
0	(0,8)	$1-r = -7$	E
1	(1,8)	-4	E
2	(2,8)	1	SE
3	(3,7)	-6	E
4	(4,7)	7	SE
5	(5,6)	...	Stop when $x \geq y$

The remaining points come from symmetry.
This is why only a small amount of computation is needed.

A simplified iteration table is:

k	Current (x, y)	Decision p	Choice	Next point in octant
0	(0, 8)	-7	E	(1, 8)
1	(1, 8)	-4	E	(2, 8)
2	(2, 8)	1	SE	(3, 7)
3	(3, 7)	-6	E	(4, 7)
4	(4, 7)	3 or 7 depending on indexing style	SE	(5, 6)

After that, we stop soon because x approaches y .

Then each octant point is reflected to the other 7 octants.

Important note

Do not panic if your decision-parameter table differs slightly from another textbook. Some books update x first and some conceptually decide before updating. The **selected pixels** are the important part.

7. Bresenham's circle algorithm

Many courses mention **Bresenham's circle algorithm** separately from the midpoint circle algorithm.

In practice, the two are extremely close in spirit.

Like Bresenham's line algorithm, the goal is:

- avoid expensive floating-point arithmetic,
- use an integer decision variable,
- update the decision variable incrementally.

So Bresenham's circle algorithm also uses:

- local neighbor choice,
- symmetry,
- small integer updates.

A common presentation is:

```
x = 0
y = r
d = 3 - 2r

while x <= y:
    plot8(x, y)
    if d < 0:
        d = d + 4x + 6
    else:
        d = d + 4(x - y) + 10
        y = y - 1
    x = x + 1
```

Midpoint vs Bresenham circle

Circle algorithms in perspective

Aspect	Midpoint circle	Bresenham circle
Core idea	Test midpoint using implicit circle equation	Use integer decision updates in Bresenham style
Arithmetic	Mostly integer additions/subtractions after setup	Also integer additions/subtractions
Teaching value	Very intuitive geometric test	Often presented as an optimized circle rasterizer
Relationship	Conceptually the basis	Practically very close / often viewed as a variant
Takeaway	Choose between E and SE in an octant	Same efficient spirit, minimal arithmetic

In many textbooks, the two are taught separately, but students should notice how similar their goals and updates are.

Conceptually:

- **midpoint circle** emphasizes the geometric test using the midpoint,
- **Bresenham circle** emphasizes integer recurrence and efficient updates.

In many textbooks, Bresenham's circle algorithm can be understood as an efficient integer form of the same basic idea.

Teaching advice

For exams and conceptual understanding, it is usually best to learn the **midpoint idea** first. Once that is clear, Bresenham's version feels natural.

8. Common mistakes in circle drawing

We often make these mistakes:

1. Forgetting to plot all 8 symmetric points.
2. Continuing too long instead of stopping near $x = y$.
3. Mixing up (x, y) and (y, x) during symmetry plotting.
4. Using the wrong sign in the decision update.
5. Confusing the midpoint circle parameter with Bresenham's line parameter.

Exam tip

If you forget the full derivation during an exam, remember the story:

We are in one octant, we choose between **E** and **SE**, and we use symmetry to finish the whole circle.

Even that story already earns conceptual credit.

9. Ellipse scan conversion — short discussion

An ellipse centered at the origin is given by:

$$x^2 / a^2 + y^2 / b^2 = 1$$

where:

- **a** = semi-major axis,
- **b** = semi-minor axis.

At first glance, ellipse drawing looks similar to circle drawing. But there is an important difference:

The slope changes differently along the curve.

 Midpoint ellipse: two regions

For this reason, the midpoint ellipse algorithm divides the first quadrant into **two regions**.

Region 1

In Region 1:

$$|dy/dx| < 1$$

The curve changes more slowly in y than in x, so we mainly advance in the x-direction.

We choose between neighboring pixels somewhat like the circle case.

Region 2

In Region 2:

$$|dy/dx| > 1$$

Now the curve changes more rapidly in y , so we mainly advance in the y -direction.

This is why ellipse scan conversion uses **two decision phases**.

Main idea we should remember

You are **not** usually expected to memorize every algebraic detail unless your instructor insists. But you should know:

1. the ellipse uses a midpoint-style method,
2. symmetry is used,
3. the first quadrant is split into two regions,
4. the stepping logic changes across those regions.

Application note

Ellipses appear in computer graphics more often than you may think:

- rounded user-interface elements,
- planetary orbits in educational graphics,
- perspective-like circular features,
- logos and typography,
- arc construction in CAD systems.

10. Scan converting a rectangle

A rectangle is much easier to rasterize than a circle or ellipse.

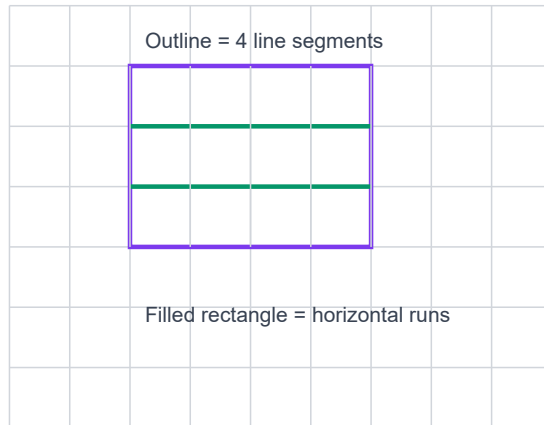
If the rectangle is axis-aligned and defined by:

```
x_min, y_min, x_max, y_max
```

then:

- its boundary is just **four line segments**,
- its filled interior is just a set of **horizontal runs** or **vertical runs**.

Scan converting a rectangle



```
for y = y_min to y_max:  
  for x = x_min to x_max:  
    plot(x, y)
```

Optimized view:

```
for each scan line y:  
  fill from x_left to x_right
```

A rectangle is one of the easiest shapes to rasterize. GUI buttons, windows, and panels all rely on this idea.

Outline rectangle

To draw the outline:

1. draw top edge,
2. draw bottom edge,
3. draw left edge,
4. draw right edge.

These can be seen as special cases of line drawing.

Filled rectangle

For every scan line `y` between `y_min` and `y_max`, fill pixels from `x_min` to `x_max`.

Pseudocode

```
for y = y_min to y_max:  
  for x = x_min to x_max:  
    plot(x, y)
```

This is simple, but graphics hardware usually optimizes it heavily. In modern systems, rectangle fills are among the most common and fastest drawing operations.

Why rectangles matter so much

Think about how many rectangles you see in one minute of computer use:

- windows,
- cards,
- buttons,
- menu panels,
- text boxes,
- images,
- dashboards,
- tables.

A huge fraction of user interfaces is ultimately based on rectangle rasterization.

11. Region filling

Sometimes the problem is not “draw the boundary” but rather:

Fill the inside of a region with a color.

This is the job of **region filling**.

Three related ideas are important:

1. **Boundary fill**
 2. **Flood fill**
 3. **Scan-line fill**
-

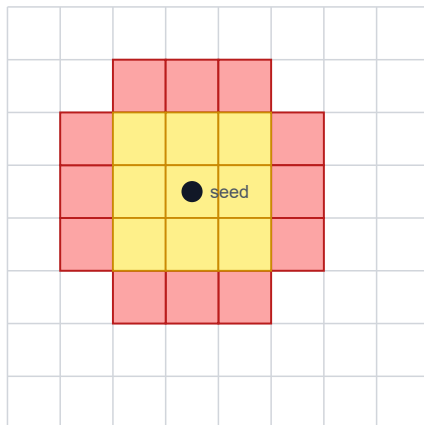
12. Boundary fill

In boundary fill, we assume the region is enclosed by a boundary of a known color.

We start from a seed point inside the region and keep filling neighboring pixels until the boundary is reached.

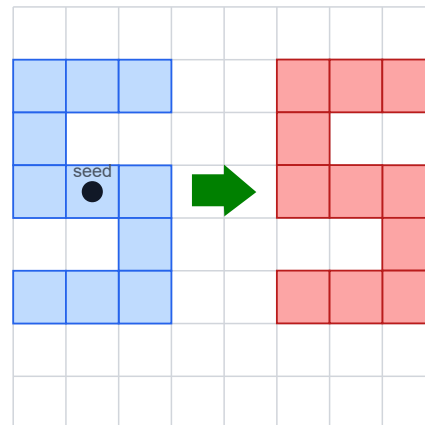
Boundary fill vs flood fill

Boundary fill



Stop when a boundary color is reached.
Useful when the border is known.

Flood fill



Replace all connected pixels of a target color.
Useful like a paint-bucket tool.

Core idea

If current pixel is not boundary color and not fill color,
fill it and recursively process neighbors.

4-connected boundary fill

```
BoundaryFill4(x, y, fillColor, boundaryColor):  
    if getPixel(x, y) is not boundaryColor and not fillColor:  
        setPixel(x, y, fillColor)  
        BoundaryFill4(x+1, y, fillColor, boundaryColor)  
        BoundaryFill4(x-1, y, fillColor, boundaryColor)  
        BoundaryFill4(x, y+1, fillColor, boundaryColor)  
        BoundaryFill4(x, y-1, fillColor, boundaryColor)
```

8-connected boundary fill

In 8-connected filling, diagonal neighbors are also processed.

This may fill regions differently from 4-connected fill.

13. Flood fill

Flood fill is similar, but the stopping rule is different.

Instead of stopping at a boundary color, flood fill replaces all connected pixels of a **target color**.

This is exactly the idea behind the **paint bucket tool** in many drawing programs.

Core idea

```
If current pixel has the old target color,  
replace it and continue to connected neighbors.
```

4-connected flood fill

```
FloodFill4(x, y, oldColor, newColor):  
    if getPixel(x, y) == oldColor:  
        setPixel(x, y, newColor)  
        FloodFill4(x+1, y, oldColor, newColor)  
        FloodFill4(x-1, y, oldColor, newColor)  
        FloodFill4(x, y+1, oldColor, newColor)  
        FloodFill4(x, y-1, oldColor, newColor)
```

Boundary fill vs flood fill

- **Boundary fill**: stop when boundary color is encountered.
- **Flood fill**: replace all connected pixels that have the old color.

A good way to remember it is:

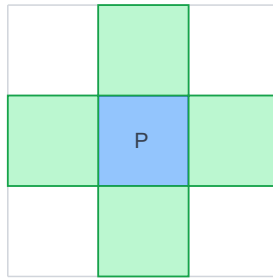
- boundary fill cares about the **border**,
- flood fill cares about the **starting region color**.

14. Connectivity: 4-connected and 8-connected

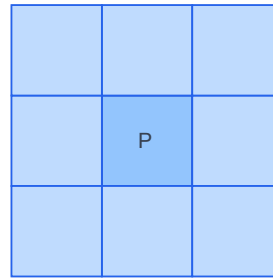
When we say “neighboring pixels”, what exactly do we mean?

4-connected and 8-connected neighborhoods

4-connected



8-connected



4-connected: only up, down, left, right

8-connected: also includes diagonal neighbors

Connectivity matters in region filling because it defines what counts as “connected”.

4-connected neighborhood

The neighbors are:

```
(x+1, y)
(x-1, y)
(x, y+1)
(x, y-1)
```

Only up, down, left, and right are included.

8-connected neighborhood

This includes the above four plus the diagonals:

```
(x+1, y+1)
(x+1, y-1)
(x-1, y+1)
(x-1, y-1)
```

Why it matters

A region that looks connected diagonally may be:

- disconnected under 4-connectivity,

- connected under 8-connectivity.

This changes the final result of the filling algorithm.

Practical note

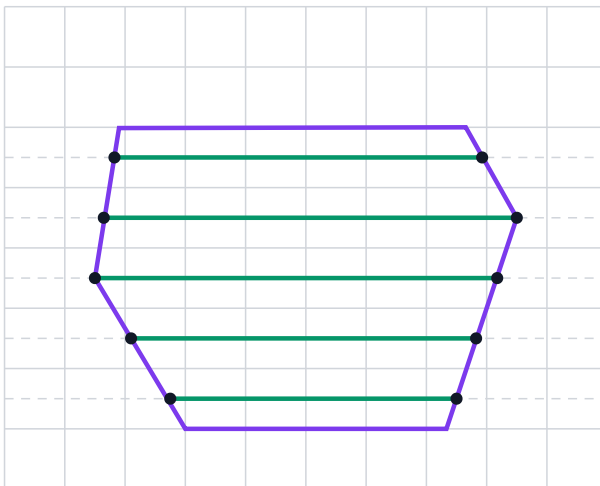
Recursive fill is simple to explain, but in real software it can cause **stack overflow** for large regions. So practical implementations often use an explicit stack or queue.

15. Scan-line fill

Boundary fill and flood fill are intuitive, but another very important filling strategy is **scan-line fill**.

Instead of expanding recursively from a seed point, scan-line fill works row by row.

Scan-line filling idea



Instead of testing pixel by pixel,
fill one horizontal run at a time.

For each scan line:

1. Find intersections with edges
2. Sort them by x
3. Fill between pairs

This is very important for polygon filling
and was widely used in software renderers.

Main idea

For each horizontal scan line:

1. find where it intersects the polygon or region boundary,
2. sort the intersection x-values,
3. fill pixels between pairs of intersections.

Why it is important

Scan-line filling is efficient and conceptually ties directly to the raster structure of the display.

It became especially important in polygon filling and software renderers.

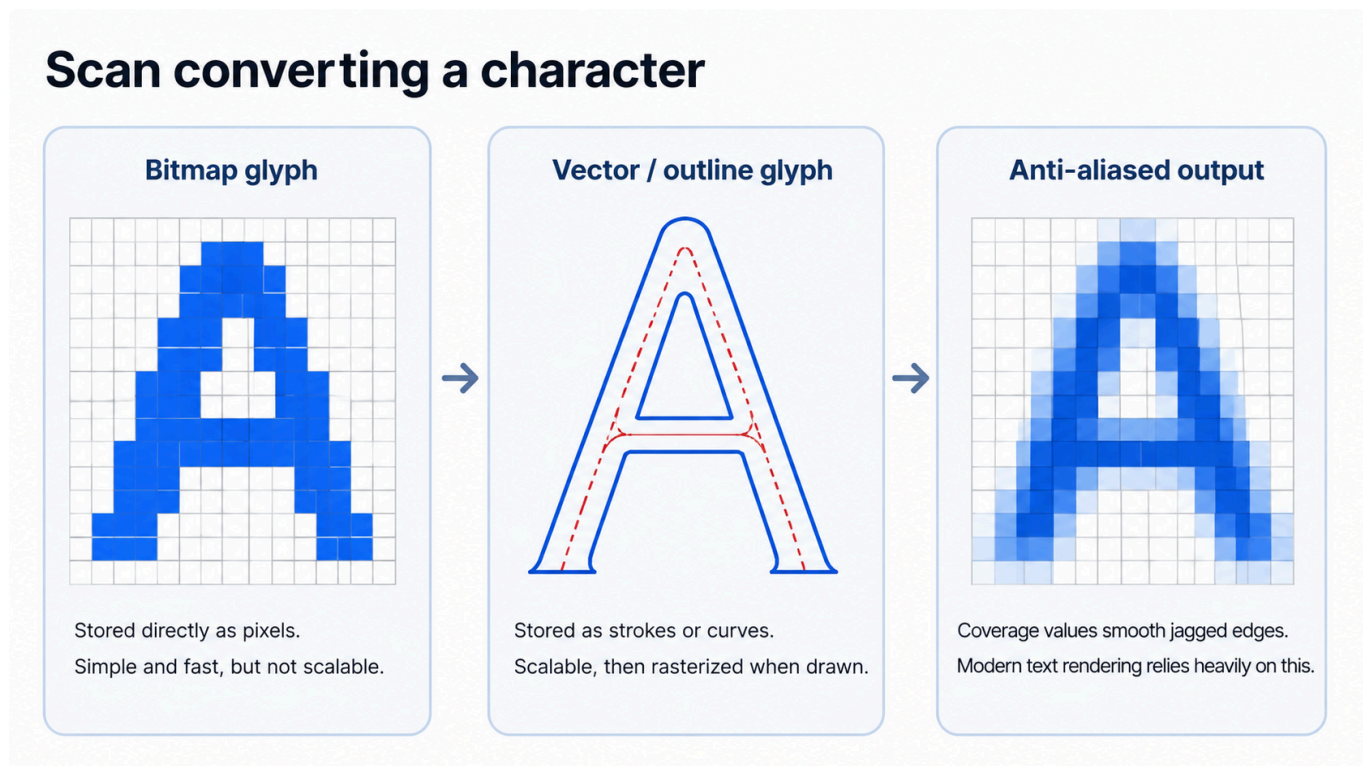
Trivia

A lot of “classic graphics textbook feeling” comes from scan-line thinking. Before modern programmable GPUs, many raster graphics systems were designed around the fact that the display is naturally processed **line by line**.

16. Scan converting a character

One of the most practical and beautiful applications of scan conversion is **text rendering**.

Every letter you see on the screen must eventually become pixels.



16.1 Bitmap characters

A bitmap font stores each character as a small pixel pattern.

Example idea:

```
A = matrix of 0s and 1s
```

Advantages:

- simple,
- fast,
- historically important.

Disadvantages:

- does not scale nicely,
- can look blocky at large sizes,
- separate versions may be needed for different sizes.

16.2 Vector or outline characters

Modern fonts usually store characters as **outlines** built from lines and curves.

Examples include TrueType and OpenType fonts.

When the system needs to display a character:

1. the outline is scaled,
2. it may be hinted or adjusted,
3. then it is rasterized into pixels.

Advantages:

- scalable,
- cleaner at many sizes,
- supports high-quality typography.

16.3 Anti-aliasing in text

Character edges often look jagged if pixels are only fully on or fully off.

So text renderers use **anti-aliasing**, meaning edge pixels may receive partial coverage or intensity values.

That is why modern text usually looks smoother than old bitmap text.

Fun trivia

Font rendering is a whole subfield of graphics and typography. Tiny details such as how the letter "e" or "g" appears at 12 pixels high involve surprising amounts of engineering.

17. Applications of this lecture

These topics are not just theory. They appear in:

- **games:** circles, mini-maps, HUD elements, health indicators,
- **CAD:** arcs, circles, hatch fill, dimensions,
- **GUI systems:** windows, buttons, rounded rectangles,
- **paint software:** bucket fill, brush interiors,
- **GIS and maps:** labeled regions, roads, symbols,
- **document systems and printers:** font rasterization,
- **embedded displays:** simple shape drawing with limited hardware.

A nice perspective

We often think the “interesting” part of computer graphics begins only in 3D. But even ordinary 2D scan conversion contains deep ideas:

- geometry,
 - efficiency,
 - approximation,
 - discrete mathematics,
 - perceptual quality.
-

18. Key comparisons to remember

Circle vs ellipse

- both use symmetry,
- both use decision-style incremental methods,
- the circle is simpler because the curvature is more uniform,
- the ellipse needs two regions.

Boundary fill vs flood fill

- boundary fill: border-defined,
- flood fill: old-color-defined.

Outline vs fill

- outline selects boundary pixels,
- filling selects the interior pixels.

Bitmap font vs outline font

- bitmap font: pixels stored directly,
 - outline font: shape stored, pixels generated later.
-

19. Short exam-style questions

1. Why is it enough to calculate only one octant in circle scan conversion?
 2. What is the role of the decision parameter in the midpoint circle algorithm?
 3. Why do ellipse algorithms use two regions?
 4. How do boundary fill and flood fill differ?
 5. What is the difference between 4-connectivity and 8-connectivity?
 6. How can a filled rectangle be generated efficiently?
 7. Why are modern fonts usually outline-based instead of bitmap-only?
-

20. Worked discussion questions for class

Question 1

Suppose you are drawing a circle and you already found one first-octant point `(4, 7)`. What are its other 7 symmetric points?

Question 2

Why is recursive flood fill attractive for teaching but dangerous for very large images?

Question 3

If a region is connected only diagonally, what happens under 4-connectivity and 8-connectivity?

Question 4

Why is rectangle rasterization so common in user interfaces?

21. Summary

Lecture summary

- **Circle** Use symmetry. Compute one octant and reflect.
- **Midpoint / Bresenham** Choose between neighboring pixels using a decision variable.
- **Ellipse** Use two regions because the slope behavior changes.
- **Rectangle** Rasterize as 4 edges or as horizontal pixel runs.
- **Region filling** Boundary fill and flood fill depend on connectivity and stopping rules.
- **Characters** Bitmap or outline glyphs are finally turned into pixels.

Big idea: raster graphics repeatedly convert continuous descriptions into discrete colored samples.

The big takeaways are:

- **Circle scan conversion** uses symmetry and a local decision rule.
- **Midpoint circle** and **Bresenham circle** are efficient incremental methods.
- **Ellipse scan conversion** is similar in spirit but needs two regions.
- **Rectangles** are easy to rasterize, especially when filled row by row.
- **Boundary fill** and **flood fill** are fundamental region-filling methods.
- **Scan-line fill** is an efficient row-wise filling idea.
- **Characters** are rasterized either from stored bitmaps or from scalable outlines.

If Lecture 3 taught us how to convert lines into pixels, Lecture 3.1 teaches us how the same philosophy extends to richer 2D objects.

22. Suggested follow-up topics

- polygon scan conversion,
 - clipping,
 - anti-aliasing,
 - 2D transformations,
 - font rendering and vector graphics,
 - the rasterization stage in modern 3D pipelines.
-

23. Final trivia box

- The same Bresenham who gave us Bresenham's line algorithm also influenced efficient raster methods for other primitives.
- Even today, "circle drawing" is not a toy problem — it appears in GUI libraries, embedded displays, CAD, and game engines.
- Text rendering may look like a "simple" part of software, but high-quality font rasterization is one of the most refined engineering achievements in 2D graphics.